

Optimizing LLM Agent Assignment under Token and Cost Budgets

Wengxin Xu

wengxin.xu@mail.utoronto.ca

Repository: <https://github.com/Wengxin04/llm-routing-optimization>

1 Introduction

Given a stream of queries and heterogeneous LLM agents with different quality, cost, latency, and token-budget tradeoffs, we (i) formulate the offline query-to-agent assignment as an integer program, (ii) study its solution under three budget scenarios, (iii) design fast methods for online deployment, and (iv) extend the framework to resource-aware real-time routing. Our main methodological contribution is an adaptive primal-dual streaming router that outperforms a batched LP-relaxation baseline by 5.6% at ~ 0.03 ms per-query decision time while remaining fully streaming.

2 Formulation

Let $I = \{1, \dots, N\}$ index queries, $J = \{A, B, C, D, E\}$ index agents, and $K = \{512, 1024, 2048, 4096\}$ index token-budget levels. The dataset provides observed quality q_{ijk} , cost c_{ijk} , and latency ℓ_{ijk} for each triple; token consumption equals the budget level, $t_{ijk} = k$. Let $x_{ijk} \in \{0, 1\}$ equal one iff query i is routed to agent j with token budget k .

$$\begin{aligned} \max \quad & \sum_{i,j,k} q_{ijk} x_{ijk} \\ \text{s.t.} \quad & \sum_{j,k} x_{ijk} = 1 \quad \forall i \in I \\ & \sum_{i,j,k} t_{ijk} x_{ijk} \leq B_T \\ & \sum_{i,j,k} c_{ijk} x_{ijk} \leq B_C \\ & \sum_{i,k} x_{ijk} \leq \text{cap}_j \quad \forall j \in J \\ & x_{ijk} \in \{0, 1\} \end{aligned} \tag{1}$$

with defaults $B_T = 1000N$, $B_C = 0.00022N$, and per-agent capacities

$$\text{cap}_A = \lfloor 0.30N \rfloor, \quad \text{cap}_{B,C,D} = \lfloor 0.25N \rfloor, \quad \text{cap}_E = \lfloor 0.20N \rfloor.$$

Assumptions. (i) Every query must be answered, forcing the equality in the first constraint. (ii) Observed quality is treated as a known constant for the offline optimizer; this assumption is relaxed by the streaming methods in Section 4. (iii) Resources are additively consumed with no queueing effects.

Problem class. This is a Multiple-Choice Multi-dimensional Knapsack Problem with generalized-assignment side constraints [1], NP-hard in general. The train instance (5,320 binary variables, 273 constraints) is nonetheless solved by CBC branch-and-bound within a 90s time limit.

3 Computational Study

We solve the integer programming (IP) on the train split ($N = 266$) using PuLP with the CBC solver under three budget scenarios: default, tight ($0.7\times$), and loose ($1.5\times$) (see Figure 1).

Table 1: Task 2 results on train, $N = 266$. Oracle = $\sum_i \max_{j,k} q_{ijk} = 218.03$.

Scenario	Objective	% oracle	Tokens used	Cost used	Solve time
Tight ($0.7\times$)	185.41	85.0%	99.8%	99.8%	90s (limit)
Default	211.88	97.2%	99.9%	99.6%	90s (limit)
Loose ($1.5\times$)	218.03	100.0%	100.0%	100.0%	0.2s

Findings. (i) The loose scenario recovers the oracle upper bound exactly. Increasing the budget beyond $\sim 1.3\times$ the default yields zero marginal quality, as spare resources are absorbed by more expensive options that do not improve the objective. (ii) The marginal quality per budget unit is roughly $5\times$ higher in the tight regime (a 14.3% quality gain for a 43% budget increase) than in the loose regime (a 2.9% gain for a 50% increase). (iii) Strict budget constraints force low-yield fallbacks: in the tight scenario, agent A absorbs 25 `math_olympiad` queries at ~ 0.20 realized quality because higher-quality agents (C and D) are completely saturated by more valuable assignments (see Figure 2).

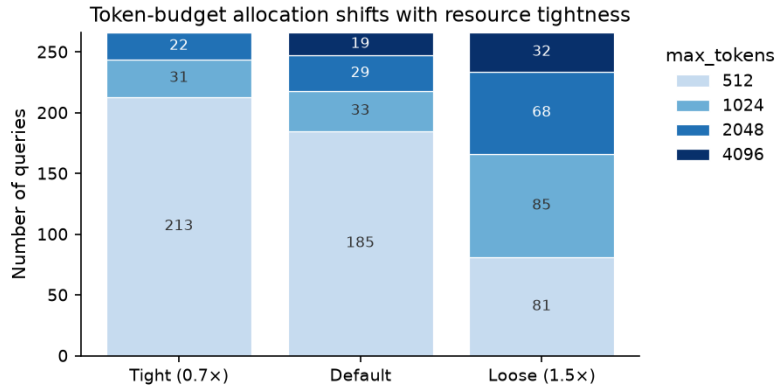


Figure 1: Token-budget allocation across scenarios. Tight uses zero 4096-token options.

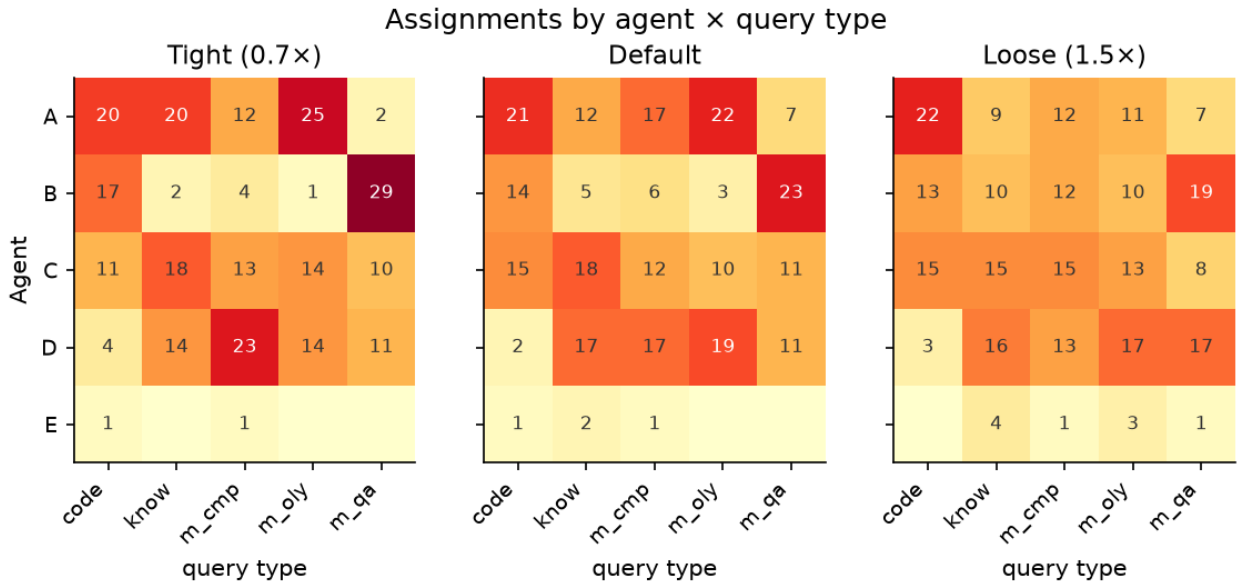


Figure 2: Assignments by agent $\times$ query type. Agents specialize, while tight budgets force agent A into low-yield fallbacks.

4 Fast Assignment Methods

The offline IP is impractical for real-time routing (taking 60–90s on the train instance). We develop a spectrum of fast methods and evaluate them on the held-out test split ($N = 109$); test q_{ijk} is consulted only for evaluation.

Rule (naive). For each query type, take the (agent, budget) with maximum train mean quality. Constraint-blind; it violates the cost budget by $6.8\times$. Included as a counter-example motivating constraint awareness.

Rule (efficiency). Rank options within each query type by train efficiency $\bar{q}/\bar{c}$; assign greedily with feasibility fallback. Serves as a basic feasible streaming baseline.

LP-Batch. Solve the LP relaxation of the routing IP over the full test batch, using train-mean quality and cost as coefficients; round via LP-weighted greedy processing (queries handled in decreasing max-LP-value order, each assigned to the argmax LP value over remaining feasible options). Requires observing the full batch before deciding.

Shadow-Static. Solve an aggregate LP on train once (offline), with continuous per-type variables. Extract shadow prices $\lambda_{\text{tok}}, \lambda_{\text{cost}}, \mu_j$ as the dual variables of the shared resources; each price is the marginal quality gain per unit relaxation of its constraint. Online, score each option and take the highest-scoring feasible one:

$$\text{score}(i, j, k) = \hat{q}_{ijk} - \lambda_{\text{tok}}k - \lambda_{\text{cost}}\hat{c}_{ijk} - \mu_j. \quad (2)$$

Streaming and $O(1)$ per query. Loses to LP-Batch because static prices reflect typical train-load tension rather than realized test-batch consumption.

Shadow-Adaptive (main proposal). Initialize prices from Shadow-Static, then update multiplicatively after each query [2, 3]:

$$\lambda_r(i+1) = \lambda_r(i) \cdot \exp(\eta \cdot (\rho_r(i) - i/N)), \quad r \in \{\text{tokens, cost, per-agent capacity}\}, \quad (3)$$

where $\rho_r(i)$ is the consumed fraction of resource r after i queries and $\eta = 0.5$ is selected by grid search on train. If a resource is consumed faster than progress its price rises and the router avoids it; symmetric on the other side.

Table 2: Task 3 results on test, $N = 109$. **Shadow-Adaptive** is our main proposal. % oracle is against the offline IP with observed test q .

Method	Quality	% oracle	Latency	Paradigm
Oracle (IP + observed q)	87.07	100.0%	seconds	reference
Rule (naive)	69.96	–	$O(1)$	infeasible
Rule (efficiency)	43.34	49.8%	$O(1)$	streaming
LP-Batch	51.89	59.6%	0.85 ms/q	batch
Shadow-Static	46.89	53.9%	0.017 ms/q	streaming
Shadow-Adaptive	54.82	63.0%	0.027 ms/q	streaming

Empirical observation. λ_{cost} stays at zero throughout: the aggregate LP returns $\lambda_{\text{cost}} = 0$ and multiplicative updates preserve it. Cost usage nonetheless lands at 94.6% and feasible: higher token budgets generate proportionally higher costs, so managing tokens implicitly manages cost. The algorithm correctly identifies that only tokens and per-agent capacities are active constraints in this instance.

5 Extension: Resource-Aware Real-Time Routing

Shadow-Adaptive assumes a known query horizon N and a static agent pool. Deployment departs on the five dimensions of the problem specification (Table 3); we extend the primal-dual framework to handle them jointly under a single mechanism.

Table 3: Mapping deployment requirements to Resource-Aware Adaptive Primal-Dual (RA-APD).

Dimension	Formalization	Method component
Workload	Poisson arrivals, unknown time-varying rate $\lambda_{\text{arr}}(t)$	Rolling-window multiplicative-weights (MW) update replaces i/N
Latency	$\ell(j, k, q_j) = \bar{\ell}_{jk} + \beta_j q_j$; SLA d_i per query	New shadow price λ_{sla} with hinge loss
Queue length	Live queue $q_j(t)$ observable at routing time	Adds a live-queue term $\mu_j \cdot q_j(t)/\text{cap}_j$ to the score
Capacity	Static per-agent capacity cap_j	Existing μ_j shadow price (unchanged)
Availability	Two-state $\alpha_j(t) \in \{0, 1\}$	Hard filter; MW auto-rebalances upon recovery

Method. We extend the Shadow-Adaptive score with a live-queue capacity term and an SLA hinge term:

$$\text{score}(i, j, k) = \hat{q}_{ijk} - \lambda_{\text{tok}} k - \lambda_{\text{cost}} \hat{c}_{ijk} - \mu_j \cdot \frac{q_j(t)}{\text{cap}_j} - \lambda_{\text{sla}} [\ell(j, k, q_j(t)) - d_i]^+. \quad (4)$$

The queue term scales the capacity price by real-time congestion: an idle agent incurs no penalty, a saturated agent incurs the full μ_j . The SLA hinge $[\cdot]^+ = \max(0, \cdot)$ penalizes only when the expected completion time exceeds the deadline.

We replace the progress term i/N with a rolling-window rate comparison, removing the known-horizon assumption:

$$\lambda_r(t + \Delta t) = \lambda_r(t) \cdot \exp\left(\eta \cdot (r_r^{\text{obs}}(t) - \bar{r}_r)\right), \quad \bar{r}_r = B_r / T_{\text{window}}, \quad (5)$$

where $r_r^{\text{obs}}(t)$ is the observed rate over the past W seconds and $\bar{r}_r$ is the target rate from the per-window budget.

Availability is a hard filter: when $\alpha_j(t) = 0$, agent j 's options are removed. On recovery, j 's load fraction lags progress, so MW shrinks μ_j until the router restores j 's load share.

Evaluation plan. Replay test queries as a Poisson stream; sample d_i from a lognormal fit; model ℓ as train mean plus queue-dependent noise. Metrics: SLA-compliant quality (primary), SLA compliance rate, budget overrun frequency, P99 decision latency. Ablations remove one component at a time (rolling-window; λ_{sla} ; availability filter). Stress scenarios: $3\times$ arrival burst, single-agent failure, tight SLA $d_i \sim \mathcal{N}(2s, 0.5s)$. Success: within 5% of Shadow-Adaptive quality under nominal load; $> 95\%$ SLA compliance under $3\times$ burst.

Reproducibility

All methods are deterministic; results reproduce exactly on the fixed train/test splits. Python 3.14.7, PuLP 3.3.2 (CBC bundled), pandas 3.0.5, numpy 2.5.3, matplotlib 3.11.2. CBC time-limit 90s on Task 1–2 instances. Reproduce all numerical claims via `python task2_solver.py` (Task 2 optimization), `task2_analysis.py` (Task 2 findings and multiplier sweep), `task2_plots.py` (figures), `task3.py` (Task 3 methods), and `task3_eta_search.py` (η grid search). **AI tools:** Claude (Anthropic) was used interactively for concept discussion, code review, and prose editing; all formulations, designs, and numerical results were verified by the author before submission.

References

- [1] S. Martello and P. Toth. *Knapsack Problems: Algorithms and Computer Implementations*. Wiley, 1990.
- [2] S. Arora, E. Hazan, and S. Kale. The multiplicative weights update method: a meta-algorithm and applications. *Theory of Computing*, 8:121–164, 2012.
- [3] N. Buchbinder and J. Naor. The design of competitive online algorithms via a primal-dual approach. *Foundations and Trends in Theoretical Computer Science*, 3(2–3):93–263, 2009.